

RTS Demo - Game Design Document

1. GAME OVERVIEW

Title: RTS Demo **Type:** Real-Time Strategy Tech Demo / Prototype **Platform:** Windows Standalone (x64) **Engine:** Unity 2022.3.62f1 **Development Status:** Prototype/Tech Demo **Scope:** Single-player sandbox skirmish between two opposing teams

Core Concept

A barebone RTS sandbox demonstrating fundamental real-time strategy mechanics: multi-unit selection, pathfinding-based movement, tactical combat, and team-based coordination. Currently supports 2 teams (Red/Blue) with extensible architecture for additional teams. Includes persistent save/load functionality.

2. CORE GAMEPLAY SYSTEMS

2.1 Unit Control & Selection

- **Box Selection:** Drag-select multiple units via BoxSelection system
- **Individual Selection:** Click units to select/deselect
- **Control Groups:** Create unit groupings via ControlGroupItem for quick recall (framework in place)
- **Visual Feedback:** Selection rings displayed on selected units (indicator prefabs)

Key Scripts:

- SelectableObject.cs - Entity base class for all selectable game objects
- Unit.cs - Unit-specific behavior (combat, movement, targeting)
- BoxSelection.cs - Drag-box selection UI/logic
- ControlGroupIndicator.cs - Visual group identification

2.2 Movement & Pathfinding

- **Navigation:** Unity AI.Navigation with NavMesh-based pathfinding
- **Agent Types:** SwiftAgent (infantry), HeavyAgent (tanks)
- **Movement Command:** Right-click destination → units pathfind via NavMeshAgent
- **Stopping Distance:** Units halt at 4m from target destination
- **Terrain:** Two terrain assets used for NavMesh generation (assets: New Terrain.asset, New Terrain 1.asset)

Key Scripts:

- MovingRange.cs - Defines movement radius and destination validation
- Unit.cs - NavMeshAgent control and pathfinding state management

2.3 Combat System

- **Target Acquisition:** Units acquire targets within AttackingRange
- **Damage Model:**
 - Base damage: 20 per shot (configurable per unit)
 - Attack cooldown: 1.0s (configurable)
 - Health: 100 HP baseline
 - Destructible: Units die when health ≤ 0
- **Damage Types:**
 - **Direct Damage:** Single-target attacks
 - **Splash Damage:** Area-of-effect attacks (configurable radius: ~5m)
- **Attack Animation:** Animator-driven firing states
- **AI Behavior:** Units automatically attack enemies in range; disengage if target dies

Key Scripts:

- AttackingRange.cs - Range detection and target validation
- Unit.cs - Combat logic, damage calculation, health management

2.4 Team System

- **Teams:** Red (Team.Red) vs Blue (Team.Blue); easily extensible to additional teams
- **Team Identity:** Controlled via Team.cs enum
- **Visual Distinction:**
 - Red materials: RedBody.mat, RedTank.mat
 - Blue materials: BlueBody.mat, BlueTank.mat
 - Team-based material assignment on prefab instantiation
- **Faction Logic:** Units only attack opposing teams; friendlies are ignored

Key Scripts:

- Team.cs - Team enumeration
- SelectableObject.cs - Team affiliation and ally/enemy checks

2.5 Unit Types

- **SwatGuy**
(Infantry)
 - Model: Humanoid soldier with rifle
 - Animations: Idle, Run, Firing, Dying
 - Attack: Direct bullet fire (no splash)
 - Prefabs: RedSwatGuy.prefab, BlueSwatGuy.prefab
- **HeavyTank**
(Vehicle)
 - Model: T4-B tank model with textured tracks
 - Attack: Single-target or splash-capable (configurable)
 - Prefabs: RedHeavyTank.prefab, BlueHeavyTank.prefab

Key Scripts:

- UnitType.cs - Unit type enumeration (Infantry, Tank, etc.)
- Unit.cs - Unit behavior polymorphism

2.6 Minimap System

- **Purpose:** Real-time tactical overview of battlefield
- **Display:** Top-right corner UI (CanvasRenderTexture-based)
- **Texture:** MinimapTexture.asset (480x480 render target)

Key Components:

- Minimap camera capturing terrain and unit positions
- Team color coding in minimap (Red/Blue unit icons)

2.7 Camera Control

- **Type:** Isometric/RTS-style free camera
- **Controls:** Keyboard pan (WASD), scroll wheel zoom
- **Boundaries:** Constrained to SampleScene terrain limits

Key Scripts:

- CameraControl.cs - Camera movement and zoom logic
-

3. PROJECT ARCHITECTURE

3.1 Entity Hierarchy

SelectableObject (Base)

 • Unit (Extends SelectableObject)

 • Animator control

 • NavMeshAgent movement

 • Attack/damage logic

 • Health system

 • Team affiliation

 • [Future: Buildings, Structures]

3.2 Manager Pattern

- **GameManager.cs** - Singleton coordinating game state, unit spawning, team initialization
- **BoxSelection.cs** - Selection state and multi-unit command dispatch
- **CameraControl.cs** - Camera state and input handling

3.3 Data Flow

1. **Input:** Player clicks/draggs → BoxSelection or CameraControl
2. **Selection:** SelectableObject.SetSelected() → updates UI, visual indicators
3. **Command:** Right-click → Unit.SetDestination() → NavMeshAgent pathfinding
4. **Combat:** Units in range → PerformAttack() → TakeDamage() → health/death handling
5. **Persistence:** Save/Load via LiteDB for game state snapshots

3.4 Team Coordination

- **Support Requests:** RequestSupportFromNearbyUnits() - Units within backupUnitRadius (~20m) respond to allies under attack
 - **Team Colors:** Materials assigned at prefab instantiation; no runtime tinting
-

4. SCENE & ENVIRONMENT

4.1 SampleScene Layout

- **Single playable scene:** Assets/Scenes/SampleScene.unity
- **Terrain:** Two terrain assets with NavMesh baked (NavMesh-Terrain.asset)
- **Foliage:** Terrain Sample Assets vegetation (bushes, ferns, grass) for visual cover
- **Lighting:** Directional light setup for HDRP/URP compatibility
- **Canvas:** World UI for health bars, selection rings, minimap

4.2 NavMesh Configuration

- **Baking:** Manual bake via Unity Navigation window
 - **Agent Types:** SwiftAgent (radius 0.4m), HeavyAgent (radius 0.6m)
 - **Obstacles:** Static terrain and foliage
 - **File:** Assets/Scenes/SampleScene/NavMesh-Terrain.asset
-

5. ASSET ORGANIZATION

5.1 Asset Packs

- **WarFX**
(v2.0+) - Professional bullet/explosion effects
 - ~100+ prefab effects across Desktop/Mobile variants
 - Modular material system for customization
 - Spawn System for pooling/instantiation
- **Cartoon FX** - Effect editing tools and supplementary particle systems
- **Terrain Sample Assets** - Foliage, terrain textures, brushes

5.2 Unity Systems

- **AI.Navigation** - NavMesh-based pathfinding (NavMeshAgent)
- **Animator** - Animation state machine control
- **UI.Slider** - Health bar rendering
- **Particle System** - Visual effects (attacks, impacts, death)
- **HDRP/URP** - High-Definition/Universal Render Pipeline support

5.3 TextMesh Pro

- Used for UI text rendering (scores, labels, debug info)
 - Liberation Sans font included
-

6. UNIT MODELS & ANIMATIONS

Assets/

 Imports/Units/

 SwatGuy/ # Infantry character

 [FBX files] # Rifle Idle, Run, Firing, Death

 [Textures] # Diffuse, Normal, Specular, Glossiness

```

    [HeavyTank/      # Vehicle model
    [M4-B.fbx
    [Textures]      # Track diffuse, normal, AO
    Animations/Units/SwatGuy/
    [SwatGuyController.controller # Animator controller
    [SwatGuyIdle.anim
    [SwatGuyRun.anim
    [SwatGuyFiring.anim
    [SwatGuyDying.anim
    Materials/Units/
    [SwatGuy/
    [RedBody.mat
    [BlueBody.mat
    [Team color variants]

```

6.2 Effects & Particles

- **WarFX Asset Pack:**
Bullet impacts, explosions, muzzle flashes, smoke
 - Desktop effects: High-quality (used for prototype)
 - Mobile effects: Optimized variants (pre-imported)
- **Cartoon FX Easy Editor:** Tools for effect editing/spawning

Key Effect Prefabs:

- WFX_BImpact Concrete.prefab - Bullet hole decals
- WFX_Explosion.prefab - Explosion with smoke
- WFX_MF 4P RIFLE1.prefab - Rifle muzzle flash

6.3 Unit Prefabs

```

Assets/Prefabs/Units/
[SwatGuy/
[SwatGuy.prefab      # Base variant (neutral)
[RedSwatGuy.prefab   # Team Red
[BlueSwatGuy.prefab  # Team Blue
[HeavyTank/
[HeavyTank.prefab    # Base variant
[RedHeavyTank.prefab # Team Red
[BlueHeavyTank.prefab # Team Blue
[Indicators/
[SwatGuy.prefab      # Selection ring indicator

```

6.4 UI Assets

```

Assets/UI/
[MinimapTexture.asset # 480x480 RenderTexture
[Sprites/
[Selection/SelectionSprite.png # Selection ring

```

☒ ☒ ☒ DestinationMarker.png # Move target marker
☒ ☒ ☒ Thumbnails/SwatGuy.png # Unit portrait
☒ ☒ [Canvas hierarchy in SampleScene]

7. UI & CONTROLS

7.1 Input Scheme

Action	Input	Result
Select Unit	Left Click	Single unit selection
Multi-Select	Drag Box	Multiple unit selection
Move	Right Click	Destination marker ☒ pathfind
Camera Pan	WASD / Arrow Keys	Free camera movement
Camera Zoom	Mouse Wheel	Zoom in/out
Deselect	Escape / Click Empty Space	Clear selection

7.2 HUD Elements

- **Health Bars:** World-space UI.Slider above each unit
- **Selection Rings:** Prefab indicators showing selected units
- **Destination Marker:** Sprite showing move target
- **Minimap:** Render texture in corner (unit positions + terrain)
- **Control Group Indicators:** Placeholder framework for hotkey groups

7.3 Visual Feedback

- **Unit States:**
 - Idle: Standing animation
 - Moving: Run animation + NavMesh locomotion
 - Attacking: Firing animation + muzzle flash effect
 - Dead: Death animation ☒ unit destroyed after 3s
 - **Selection State:** Green outline/ring on selected units
-

8. BUILD & DEVELOPMENT

8.1 Build Configuration

- **Target:** StandaloneWindows64
- **Scene:** Assets/Scenes/SampleScene.unity (only scene in build)
- **Output:** RTS demo.exe (configurable in Build Settings)

8.2 Build Steps

1. **Unity Editor:** Open project in Unity 2022.3.62f1
2. **Verify NavMesh:** Window ▢ AI ▢ Navigation ▢ Bake (if terrain modified)
3. **Build:** File ▢ Build Settings ▢ Add Scene ▢ Build

8.3 Development Workflow

- **Scene Setup:** All gameplay in SampleScene.unity
- **Prefab Editing:** Edit unit prefabs directly or via Prefab Mode
- **Testing:** Play mode with 2-3 unit spawns to verify pathfinding/combat
- **NavMesh Debugging:** Scene view toggle "Show NavMesh" for validation

8.4 Performance Targets

- **Frame Rate:** 60 FPS (Windows 64-bit)
- **Unit Capacity:** 50-100 units per team (depending on hardware)
- **Draw Calls:** Minimized via material instancing (team colors)

9. CRITICAL CONVENTIONS

9.1 Naming Conventions

Element	Format	Example
Unit Prefabs	[Color][Type].prefab	RedSwatGuy.prefab, BlueHeavyTank.prefab
Materials	[Type] or [Color][Type].mat	RedBody.mat, BlueTank.mat
Scripts	PascalCase, singular nouns	Unit.cs, SelectableObject.cs
Enums	File = PascalCase, values = PascalCase	Team.Red, UnitType.Infantry
Effects	WFX_[Effect].prefab or CFX_[Effect].prefab	WFX_Explosion.prefab

9.2 Team System

- **Enum Values:** Red = 0, Blue = 1 (extensible to Green = 2, etc.)
- **Prefab Variants:** Always provide color-specific prefabs (never runtime tinting)

- **Material Assignment:** Swapped at instantiation via GameManager or prefab variants
- **Ally Check:** `unit.selectableObject.team == currentTeam` before damage/support

9.3 Unit Prefab Structure

[Unit Root]

- ☒ ☒ **M**esh/Model (with Team-colored Material)
- ☒ ☒ **C**olliders (Box/Capsule for selection)
- ☒ ☒ **N**avMeshAgent (configured for agent type)
- ☒ ☒ **A**nimator (linked to controller)
- ☒ ☒ **U**nit (script - configurable stats)
- ☒ ☒ **S**electableObject (script - team, type)
- ☒ ☒ **A**ttackingRange (collider-based range detection)
- ☒ ☒ **M**ovingRange (reference for validation)
- ☒ ☒ **H**ealthBar UI (Canvas/Slider)
- ☒ ☒ **E**ffects (ParticleSystems for attack/impact/death)

9.4 Effect Spawning Conventions

- **Muzzle Flashes:** Instantiate at weapon muzzle point via `shootingEffect` reference
- **Hit Effects:** Spawn at impact location via `hitEffect` or surface-specific impact prefab
- **Death Effects:** Persist 3s via `destroyDeathEffectAfterSeconds` before cleanup
- **Splash Damage:** Use `Physics.OverlapSphere(position, radius)` to find targets

9.5 Script Organization

Assets/Scripts/

- ☒ ☒ **[C**ore Managers]
- ☒ ☒ ☒ **G**ameManager.cs
- ☒ ☒ ☒ **B**oxSelection.cs
- ☒ ☒ ☒ **C**ameraControl.cs
- ☒ ☒ **[E**ntity Systems]
- ☒ ☒ ☒ **U**nit.cs
- ☒ ☒ ☒ **S**electableObject.cs
- ☒ ☒ ☒ **M**ovingRange.cs
- ☒ ☒ ☒ **A**ttackingRange.cs
- ☒ ☒ **[U**I]
- ☒ ☒ ☒ **C**ontrolGroupIndicator.cs
- ☒ ☒ ☒ **[M**inimap logic embedded in Canvas]
- ☒ ☒ **E**nums/
- ☒ ☒ ☒ **T**eam.cs
- ☒ ☒ ☒ **U**nitType.cs
- ☒ ☒ ☒ **S**electableType.cs
- ☒ ☒ **S**tructures/
- ☒ ☒ ☒ **C**ontrolGroupItem.cs

9.6 Common Workflows

Adding a New Unit Type:

1. Create model in Assets/Imports/Units/[Type]/
2. Set up Animator controller in Assets/Animations/Units/[Type]/
3. Create base prefab: Assets/Prefabs/Units/[Type]/[Type].prefab
4. Duplicate for teams: Red[Type].prefab, Blue[Type].prefab
5. Add UnitType.[Type] enum value
6. Configure Unit.cs serialized fields (health, damage, ranges)

Testing Combat:

1. Spawn 2-3 Red units, 2-3 Blue units in SampleScene
2. Position near each other (within 10m)
3. Verify: Attack animation ☐ effects ☐ damage numbers ☐ death
4. Check minimap reflects unit positions/colors

Expanding Teams:

1. Add Green = 2 to Team.cs enum
2. Create material variants: GreenBody.mat, GreenTank.mat
3. Duplicate prefabs: GreenSwatGuy.prefab, GreenHeavyTank.prefab
4. Update GameManager spawn logic to handle Team.Green

10. KNOWN LIMITATIONS & FUTURE EXPANSION

10.1 Current Prototype Scope

- ☐ 2-team skirmish only (extensible to N teams)
- ☐ 2 unit types (SwatGuy, HeavyTank)
- ☐ No economy, production, or base building
- ☐ No win/loss conditions
- ☐ Single-scene sandbox

10.2 Expansion Potential

- **Teams:** Trivial (add enum, materials, prefabs)
- **Unit Types:** Add to UnitType enum, create prefabs, configure stats
- **AI Behavior:** Implement IAgent interface for autonomous unit control
- **Base Building:** Extend SelectableObject for static structures
- **Economy:** Resource manager + production queues